Assumptions:

. No lateral slipping (non-holonomic constraint)
. Motion is planar (2D only)
. Perfect rolling without wheel slip
. Instantaneous control inputs (no delay)
. Rigid body robot assumption
. Ideal actuators and sensors (for modeling)
. Known inertial reference frame

1-Robot Kinematics & ENCODER ODOMETRY

Input: ωr, ωl
Output: x, y, θ
Constants: r (Radius of wheels) , L (distance between the 2 wheels)

*Linear velocity:

$$v=(R/2)*(\omega r+\omega l)$$

*Angular velocity:

$$\omega=(R/L)*(\omega r-\omega l)$$

*Change in direction:

$$\theta_{new} = \theta_{old} + \omega \cdot dt$$

*Change in position:

$$X_{new} = X_{old} + V\cos(\theta)\cdot dt$$
$$Y_{new} = Y_{old} + V\sin(\theta)\cdot dt$$

Robot Model: Differential Drive

Parameters:

· r: wheel radius (m)
.L : distance between the two wheels (track width) (m)
· ωR: angular velocity of the right wheel (rad/s)
· ωL: angular velocity of the left wheel (rad/s)
· v : linear velocity of the robot (m/s)

· ω : angular velocity of the robot (rad/s)

Forward Kinematics

2.1 Derivation

The linear velocity of each wheel is:

$$vR = r * \omega R$$

$$vL = r * \omega L$$

The robot's overall linear velocity is the average of the two wheel velocities:

$$v = \frac{vR + vl}{2} = \frac{r\omega R + r\omega L}{2}$$

$$v = \frac{r}{2}(\omega R + \omega L)$$

The robot's angular velocity is derived from the difference in wheel velocities divided by the wheel track width:

$$\omega = \frac{vR - vL}{L} = \frac{r\omega R - r\omega L}{L}$$

$$\omega = \frac{r}{L}(\omega R - \omega L)$$

2.2 Position Update

Given the robot's previous pose $(xt, yt, \theta t)$ and time step Δt, the new pose is:

$$x(t+1) = xt + v * \cos\theta t * \Delta t$$
$$y(t+1) = yt + v * \sin\theta t * \Delta t$$
$$\theta(t+1) = \theta t + \omega * \Delta t$$

Inverse Kinematics

Purpose: Given desired robot velocities (v, ω), compute the required angular velocities for each wheel.

Starting from the forward kinematics equations:

$v = \frac{r}{2}(\omega R + \omega L)$ ..... $\omega R + \omega L = \frac{2v}{r}$ & $\omega = \frac{r}{L}(\omega R - \omega L)$ ..... $\omega R - \omega L = \frac{L\omega}{r}$

Solving the system of two equations:

1. Add them:

$$2\omega R = \frac{2v}{r} + \frac{L\omega}{r} = \frac{2v + L\omega}{r}$$
$$\omega R = \frac{2v + L\omega}{2r}$$

2. Subtract:

$$2\omega L = \frac{2v}{r} - \frac{L\omega}{r} = \frac{2v - L\omega}{r}$$
$$\omega L = \frac{2v - L\omega}{2r}$$

LINE CONTROLLER

The line following controller is constructed by:

1. Estimating line position using a weighted sensor model
2. Computing tracking error relative to a reference center
3. Applying a PID control law to generate steering command
4. Mapping control output to differential wheel velocities
5. Adjusting forward speed based on tracking error magnitude

$$e(t) = x_{desired} - x_{measured}$$

- $e(t) = 0$: robot is centered on the line
- $e(t) > 0$: deviation to one side

- $e(t) < 0$: deviation to the opposite side

2-PID Controller Derivation

Objective:

To generate a smooth control signal based on present, past, and predicted error behavior.

Proportional Term (P)

$$P \propto e(t)$$
$$P = K_p e(t)$$

Integral Term (I)

The integral term accounts for accumulated past error:

$$I \propto \int e(t)\, dt$$
$$I = K_i \int e(t)\, dt$$

Derivative Term (D)

The derivative term predicts future error trend:

$$D \propto \frac{de(t)}{dt}$$
$$D = K_d \frac{de(t)}{dt}$$

Final PID equation:

$$u(t) = K_p e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt}$$

Interpretation:

- **P term:** reacts to current error
- **I term:** eliminates steady-state error
- **D term:** reduces oscillations and improves stability

3-Differential Drive Kinematics

Objective:

To convert the control signal into individual wheel velocities.

Assumptions:

- v: linear (forward) velocity
- ω: angular velocity (steering control output)

Derivation:
To achieve rotation, wheel speeds are adjusted asymmetrically:

$$V_R = v + \omega$$
$$V_L = v - \omega$$

Interpretation:
- If $\omega > 0$: robot turns left
- If $\omega < 0$: robot turns right
- If $\omega = 0$: robot moves straight

4-Speed Adaptation Based on Error

Objective:
To reduce speed during sharp turns for stability.

Equation:

$$v = v\_base\ (1 - |e|) + (Vmin * |e|)$$

Parameters:
- v_{base}: maximum forward speed
- $Vmin$: minimum speed
- $|\ e\ |$: absolute tracking error

Interpretation:
- Low error → high speed
- High error → reduced speed for stability

Related Topics

1-Linear Scaling:

Definition

Linear Scaling is used to convert values from one range into another range while preserving the proportional relationship between values.

For example:
• Sensor readings may range from 0 to 1023.
• Motor speed may require values from 0 to 255.
Therefore, scaling is required to map the sensor values to suitable motor speed values.

Equation

$$y = \frac{(x - x_{min})(y_{max} - y_{min})}{x_{max} - x_{min}} + y_{min}$$

Parameters:

x: Original input value (sensor reading)
$x_{min:}$ Minimum value of the original range
x_{max}: Maximum value of the original range
y: New scaled output value
y_{min}: Minimum value of the new range
y_{max}: Maximum value of the new range

Example

Given:

- Sensor range: 0 → 1023
- Motor speed range: 0 → 255
- Sensor reading: $x = 700$

Substitute into the equation:

$$y = \frac{(700 - 0)(255 - 0)}{1023 - 0}$$

Result:

$$y \approx 174$$

So, the sensor reading 700 is converted into a motor speed of approximately 174.

In the line follower robot, linear scaling is used to convert sensor or control values into PWM signals suitable for motor control.

2- Angle Normalization

Definition

Angle Normalization is used to keep angles within a specific range.
In robotics, angles may become:

- 370°
- -540°
- 725°

These angles represent the same physical direction as smaller angles. Therefore, normalization simplifies calculations.
Common angle ranges:

- -180° to 180°
- $-\pi$ to π

Equation

$$\theta norm = atan2(sin\theta, cos\theta)$$

Parameters:
θ: Original angle
θ_{norm}: Normalized angle

Example
Given:
$\theta = 390^\circ$

Substitute:

$$sin\ (390^\circ) = 0.5$$
$$cos\ (390^\circ) = 0.866$$
$$\theta_{norm} = atan2\ (sin\ \theta, cos\ \theta)$$

$\theta_{norm} = atan2(0.5, 0.866)$
Result:

$$\theta_{norm} = 30^\circ$$

3- Min-Max Scaling

Definition

Min-Max Scaling is a normalization technique commonly used in robotics and machine learning. It transforms values into a fixed range, usually from 0 to 1.

Equation

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Parameters:
x: Original value
x_{min}: Minimum value in the dataset
x_{max}: Maximum value in the dataset
x': Normalized value

Example
Given:

- $x = 30$
- $x_{min} = 10$
- $x_{max} = 50$

Substitute:

$$x' = \frac{30 - 10}{50 - 10}$$

Result:

$$x' = 0.5$$

The normalized value is 0.5.

---

4. Euler to Quaternion Conversion

Definition

Euler angles describe orientation using three rotations:

- Roll around the X-axis
- Pitch around the Y-axis
- Yaw around the Z-axis

However, Euler angles may suffer from a problem called Gimbal Lock. To avoid this issue, robotics systems often use Quaternions.
A Quaternion is represented as:

$$q = (w, x, y, z)$$

Quaternion Conversion Equations

Scalar Component

$$q_w = \cos\left(\frac{r}{2}\right)\cos\left(\frac{p}{2}\right)\cos\left(\frac{y}{2}\right) + \sin\left(\frac{r}{2}\right)\sin\left(\frac{p}{2}\right)\sin\left(\frac{y}{2}\right)$$

X Component

$$q_x = \sin\left(\frac{r}{2}\right)\cos\left(\frac{p}{2}\right)\cos\left(\frac{y}{2}\right) - \cos\left(\frac{r}{2}\right)\sin\left(\frac{p}{2}\right)\sin\left(\frac{y}{2}\right)$$

Y Component

$$q_y = \cos\left(\frac{r}{2}\right)\sin\left(\frac{p}{2}\right)\cos\left(\frac{y}{2}\right) + \sin\left(\frac{r}{2}\right)\cos\left(\frac{p}{2}\right)\sin\left(\frac{y}{2}\right)$$

Z Component

$$q_z = \cos\left(\frac{r}{2}\right)\cos\left(\frac{p}{2}\right)\sin\left(\frac{y}{2}\right) - \sin\left(\frac{r}{2}\right)\sin\left(\frac{p}{2}\right)\cos\left(\frac{y}{2}\right)$$

Parameters:

r:Roll angle
p: Pitch angle
y: Yaw angle
qw: Quaternion scalar component
qx: Quaternion X component
qy:Quaternion Y component
q_z:Quaternion Z component